COLLEGE VIVA & ENGINEERING MASTERY GUIDE

MAPD AI MOCK INTERVIEW 2.0

Complete Project Guide from Scratch to Deployment

An in-depth, beginner-friendly technical guide explaining the complete architecture, source code, reactive database, multimodal vision/speech telemetry, AI models, and deployment workflows.

Project Name: AI Mock Interview 2.0 (MAPD)

Framework: Next.js 15.4 (App Router) + React 19

Database: Convex Serverless Reactive Document DB

Authentication: Clerk Managed Auth with JWT Middleware

AI Engine: Google Gemini 1.5 Flash + Rule Engine

Media & Vision: Canvas 2D + Web Speech + Multi-Tier Storage

Author / Student: Software Engineering Viva Candidate

Document Status: Complete Master Guide (Chapters 1–19)

TABLE OF CONTENTS

1. Project Introduction & Problem Statement CH 1
2. Complete Technology Stack Reference CH 2
3. Building the Project from Scratch CH 3
4. Complete Folder & File Structure CH 4
5. Frontend Architecture from Basics CH 5
6. Backend & Serverless Business Logic CH 6
7. Database Deep Dive (Convex Schema) CH 7
8. Complete REST API Documentation CH 8
9. API Keys & Environment Variables CH 9
10. Authentication & System Security CH 10
11. Artificial Intelligence Integration CH 11
12. Recording, Camera, Mic & Vision Telemetry CH 12
13. Complete End-to-End Execution Trace CH 13
14. Running Locally on Windows (Step-by-Step) CH 14
15. Production Deployment from Scratch CH 15
16. Testing, Debugging & Troubleshooting CH 16
17. Important Source Code Explanations CH 17
18. Viva & Interview Preparation (100+ Q&A) CH 18
19. Final Revision & Quick Reference Cheat Sheet CH 19

CHAPTER 1: PROJECT INTRODUCTION & PROBLEM STATEMENT

1.1 What is MAPD AI Mock Interview?

MAPD AI Mock Interview 2.0 is an intelligent, full-stack web application designed to help job seekers practice realistic technical and behavioral job interviews. Built with modern web standards, the platform simulates an actual hiring manager interview by combining voice interaction, real-time posture/eye-contact feedback, adaptive artificial intelligence follow-ups, and transcript-grounded performance evaluation.

The acronym **MAPD** stands for *Mock AI Placement & Development*. It transforms passive interview preparation (such as memorizing static textbook answers) into an active, immersive simulation where candidates speak out loud to an AI interviewer persona, receive follow-up probes based on their exact answers, and obtain an actionable performance scorecard graded against standard industry benchmarks like the **STAR methodology** (Situation, Task, Action, Result).

1.2 Why is it Needed? What Problem Does it Solve?

Engineering students and job seekers face significant hurdles when preparing for technical placements:

- **High Cost of Professional Coaching:** Human mock interview services (like Interviewing.io or Pramp) charge between $100 and $250 per hour, making frequent practice inaccessible for most college students.
- **Passive Learning Inefficacy:** Reading code or memorizing questions does not prepare candidates for the pressure of speaking fluently, maintaining composure, and explaining technical architectures under time constraints.
- **Lack of Objective, Immediate Feedback:** Peer practice often results in subjective, unhelpful advice ("You did okay"). Candidates rarely learn their exact filler-word count (saying *"um"*, *"like"*, *"you know"*), technical depth score, or how to structure answers using the STAR method.
- **Generic, One-Size-Fits-All Questions:** Most online quizzes ask generic questions regardless of whether the candidate is applying for a Frontend Developer, Backend Spring Boot Engineer, DevOps Specialist, or Data Scientist.

> 💡 **Viva Voice Pitch (Memorize this for your examiners):**
>
> *"My project is an AI Mock Interview platform that bridges the gap between passive learning and real-world placement interviews. It dynamically generates an 8-phase structured technical interview based on the candidate's target job role and resume, conducts the session in real-time using browser speech synthesis and recognition, monitors camera posture and eye-contact, asks adaptive follow-ups using Google Gemini 1.5 Flash, and delivers a zero-hallucination scorecard with verbal filler analysis and model STAR answers."*

1.3 Who Can Use This System?

1. **College Students & Freshers:** Preparing for on-campus placements, building verbal confidence, and practicing entry-level software engineering fundamentals.
2. **Experienced Engineers:** Practicing system design scenarios, cloud architecture questions, and behavioral leadership rounds.
3. **College Placement Cells & Bootcamps:** Automating large-scale student placement readiness assessments without needing hundreds of human evaluators.
4. **Technical Recruiters:** Reviewing recorded candidate interviews and automated scorecards before scheduling final round panel interviews.

1.4 Implemented Features vs. Planned Extensions

Feature Area	Implemented & Verified in Source Code	Implementation Details / Source Location	Status
Authentication	Clerk OAuth (Google / Email) + Session Tokens	`middleware.ts` , `app/Provider.tsx`	**Fully Implemented**
Role-Aware Questions	13 Engineering Domains + 8-Phase Progression Engine	`lib/roleInterviewerEngine.ts`	**Fully Implemented**
Resume Parsing	PDF Upload to ImageKit Cloud Storage with unique ID	`app/api/generate-interview-questions/route.tsx`	**Fully Implemented**
Voice Persona (TTS)	Male / Female Speech Synthesis via Web Speech API	`utils/interviewerConfig.ts` , `start/page.tsx`	**Fully Implemented**
Speech-to-Text (STT)	Continuous Real-time Speech Recognition in browser	`window.webkitSpeechRecognition` in `start/page.tsx`	**Fully Implemented**
Adaptive AI Probing	Google Gemini 1.5 Flash REST API + Keyword Fallback	`app/api/generate-followup-question/route.tsx`	**Fully Implemented**
Vision & Posture	Canvas 2D skin centroid & luminance sampling (<2ms)	`utils/facialAnalysis.ts`	**Fully Implemented**
Dual-Tier Recording	IndexedDB offline buffer + Supabase / Convex cloud storage	`utils/recordingStorage.ts` , `upload-recording/route.ts`	**Fully Implemented**
Transcript Feedback	Deterministic scoring: Tech, Relevance, Fillers, STAR	`app/api/interview-feedback/route.tsx`	**Fully Implemented**
Synchronized Playback	Timestamped question jump controls on recorded video	`app/(routes)/interview/[interviewId]/playback/page.tsx`	**Fully Implemented**
Streaming Video Avatar	Akool WebRTC Live Avatar SDK endpoints	`app/api/akool-session/route.ts`	**Partially Configured** (Optional)

1.5 Benefits & Current Limitations

Core Engineering Strengths:

- **Zero Latency Speech & Vision:** Because speech recognition, text-to-speech, and face tracking run inside the user's browser, there is zero per-minute audio streaming cost and no server network lag.
- **Zero-Hallucination Feedback:** The post-interview evaluation uses deterministic transcript matching rather than trusting a black-box LLM to make up scores. If the candidate never spoke a keyword, it is flagged as missing.
- **High Reliability with Fallback Systems:** If the Gemini API key runs out of quota, the system falls back to a deterministic 13-domain role engine without breaking the user's interview.

Honest Academic Limitations:

- **Browser Dependency:** Speech recognition relies on Chrome's `webkitSpeechRecognition` engine. Firefox does not natively support this API.
- **Simplified Vision Heuristics:** The posture tracking uses fast HTML5 Canvas 2D RGB skin pixel centroids rather than a heavy neural network like MediaPipe or OpenCV.

CHAPTER 2: COMPLETE TECHNOLOGY STACK REFERENCE

A critical requirement of a software engineering viva is demonstrating precise knowledge of every library and tool used in your application. Below is the verified technology stack table followed by an exhaustive breakdown of each tool.

Technology	Exact Version	Category	Primary Responsibility in MAPD	Alternative
Next.js	`15.4.10`	Full-Stack Framework	App Router pages, Server Components, Serverless API Routes	Remix, Vite + Express
React	`19.1.0`	UI Library	Declarative component rendering, state hooks, and virtual DOM	Vue.js, Svelte, Angular
TypeScript	`^5`	Programming Language	Static type safety, schema contracts, compile-time error catching	Plain JavaScript
Convex DB	`^1.25.4`	Reactive BaaS Database	WebSocket real-time document sync, session persistence, queries	Firebase Firestore, Supabase
Clerk	`^6.39.7`	Auth & User Management	JWT session tokens, Google/Email OAuth, route-guard middleware	NextAuth.js, Firebase Auth
Google Gemini	`1.5-flash`	Generative AI LLM	Real-time adaptive technical follow-up questions via REST API	OpenAI GPT-4o-mini
ImageKit	`^6.0.0`	Cloud Media Storage	Uploading and hosting candidate PDF resumes and media fallback	AWS S3, Cloudinary
Supabase Storage	`^2.53.0`	Object Storage	Hosting long interview video recordings in cloud buckets	AWS S3, Google Cloud Storage
Arcjet	`^1.0.0-beta.10`	Application Security	Token-bucket rate limiting and bot protection on API endpoints	Upstash Redis Rate Limit
Tailwind CSS	`^4`	CSS Framework	Utility-first responsive layouts, modern gradients, dark mode	Bootstrap, Styled Components
Radix UI / Shadcn	`1.1 - 1.2`	UI Primitives	Accessible unstyled modal dialogs, tabs, and button components	Material UI, Chakra UI
Web Speech API	Native Browser	Speech Synthesis & STT	Zero-cost real-time speech-to-text and interviewer persona voice	OpenAI Whisper, ElevenLabs
Canvas 2D API	Native Browser	Computer Vision Proxy	Real-time webcam pixel frame sampling for posture & eye contact	MediaPipe, OpenCV.js
IndexedDB	Native Browser	Client-Side Database	Local persistent video chunk buffer to prevent offline data loss	LocalStorage, SessionStorage

2.1 Five-Point Deep-Dive for Every Technology

1. Next.js 15 (App Router)

- **What it is:** A production React framework by Vercel offering server-side rendering (SSR), static site generation (SSG), and integrated serverless API routes.
- **Why this project uses it:** It combines both the frontend React client and the backend serverless endpoints in one unified TypeScript repository, completely eliminating cross-origin CORS errors.
- **Which files use it:** `app/layout.tsx`, `app/page.tsx`, `app/(routes)/*`, and all API routes under `app/api/*`.
- **Real-world analogy:** A modern smartphone operating system that runs user apps on screen while seamlessly handling background network sync in the background.

2. Convex Serverless Database

- **What it is:** A modern reactive document database designed specifically for TypeScript applications that streams real-time updates over WebSockets.
- **Why this project uses it:** In traditional databases, if an interview status updates, the frontend must continuously poll the server via HTTP. In Convex, the frontend hook `useQuery(api.Interview.GetInterviewQuestions)` re-renders automatically the instant any record changes.
- **Which files use it:** `convex/schema.ts`, `convex/Interview.ts`, `convex/users.ts`.
- **Real-world analogy:** A shared Google Doc where edits made by one person appear immediately on everyone's screen without clicking "Refresh".

3. Clerk Authentication

- **What it is:** A managed user identity and access management (IAM) service.
- **Why this project uses it:** It provides pre-built, secure login/signup dialogs, handles password hashing, OAuth callbacks, and verifies session tokens through Next.js middleware.
- **Which files use it:** `middleware.ts`, `app/Provider.tsx`, `app/(auth)/*`.
- **Real-world analogy:** A high-security building keycard badge that automatically opens authorized doors and keeps unauthorized guests outside.

4. Google Gemini 1.5 Flash

- **What it is:** Google's high-speed multimodal generative AI model optimized for low-latency reasoning and conversational probing.
- **Why this project uses it:** During an interview, the candidate is waiting for the next question. Gemini 1.5 Flash returns responses in under 800 milliseconds via HTTP POST, allowing natural back-and-forth conversational rhythm.
- **Which files use it:** `lib/roleInterviewerEngine.ts`, `app/api/generate-followup-question/route.tsx`.
- **Real-world analogy:** A sharp, attentive hiring manager who catches a specific keyword in your answer (like *"Docker"*) and immediately asks, *"How did you optimize your Docker container size?"*

5. Web Speech API (Synthesis & Recognition)

- **What it is:** The native JavaScript browser API for voice synthesis (TTS) and speech-to-text recognition (STT).
- **Why this project uses it:** It requires zero paid third-party API keys, runs directly on candidate device hardware, and delivers instant verbal transcription.
- **Which files use it:** `app/(routes)/interview/[interviewId]/start/page.tsx`, `utils/interviewerConfig.ts`.
- **Real-world analogy:** The human ears and vocal cords built directly into the candidate's browser window.

6. HTML5 Canvas 2D Vision Telemetry

- **What it is:** A low-level browser graphics API that allows reading and manipulating pixel arrays from video elements.

- **Why this project uses it:** By downsampling the webcam stream to 320×240 pixels and calculating skin-tone coordinate centroids, the app measures candidate posture stability and head centering every 1000ms with negligible CPU consumption.
- **Which files use it:** `utils/facialAnalysis.ts`.
- **Real-world analogy:** A virtual interview coach sitting beside you, gently tapping your shoulder if you slouch or look away from the camera.

CHAPTER 3: BUILDING THE PROJECT FROM SCRATCH

If an interviewer asks you: *"Walk me through how you built this application from a completely empty directory"*, this chapter gives you the exact chronological engineering sequence.

3.1 Step 1: Development Environment Setup

Before writing any code, install the required runtimes:

```
# 1. Verify Node.js (v18.0.0 or higher required; project uses v24)
node -v

# 2. Verify npm (Node Package Manager)
npm -v

# 3. Verify Git version control
git --version
```

3.2 Step 2: Initialize Next.js 15 Application

```
# Create the initial project with TypeScript, Tailwind CSS, and App Router
npx create-next-app@latest ai-mock-interview --typescript --tailwind --eslint --app --src-dir=false --import-alias="@/

# Navigate into the project folder
cd ai-mock-interview
```

3.3 Step 3: Install Core Dependencies

Install the exact dependencies specified in `package.json`:

```
# Install Authentication, Database, and UI Primitives
npm install @clerk/nextjs convex @supabase/supabase-js imagekit axios @arcjet/next @arcjet/inspect

# Install Icons and UI Animation
npm install lucide-react @tabler/icons-react motion next-themes sonner

# Install Radix UI components for accessible modals and tabs
npm install @radix-ui/react-dialog @radix-ui/react-tabs @radix-ui/react-slot class-variance-authority clsx tailwind-me
```

3.4 Step 4: Configure Database & Environment

```
# Initialize Convex reactive database
npx convex dev

# This automatically logs you into Convex, provisions a cloud database,
# and generates the convex/_generated/ directory with TypeScript types.
```

3.5 Step 5: Chronological Development Workflow

1. **Phase 1 — Schema & Authentication:** Set up `convex/schema.ts` defining `UserTable` and `InterviewSessionTable`. Wrap the application in `ConvexClientProvider.tsx` and `middleware.ts` to protect routes.
2. **Phase 2 — Question Engine (Domain Logic):** Create `lib/roleInterviewerEngine.ts` containing the 13 role normalizers and 8-phase question progressions.
3. **Phase 3 — Interview Configuration Dialog:** Build `CreateInterviewDialog.tsx` with role inputs, experience selection, PDF resume upload via ImageKit, and session saving into Convex.
4. **Phase 4 — Live Interview Room Engine:** Implement `start/page.tsx` connecting camera preview, speech synthesis, microphone transcription, question timer, and Canvas 2D posture tracking.
5. **Phase 5 — Adaptive AI & Feedback Evaluation:** Build the serverless endpoints `api/generate-followup-question` (Gemini 1.5 Flash) and `api/interview-feedback` (scoring & STAR answers).
6. **Phase 6 — Synchronized Playback & Recruiter Reports:** Construct the video playback timeline with question jumping and executive recruiter scorecards.

CHAPTER 4: COMPLETE FOLDER & FILE STRUCTURE

4.1 Complete Visual Architecture Tree

```
c:/Users/Deekshitha P/Documents/AI-Mock-Interview-2.0-main/
├── app/                              # Next.js 15 App Router Root
│   ├── (auth)/                       # Authentication Route Group (isolated layout)
│   │   ├── sign-in/[[...sign-in]]/   # Clerk Hosted Sign-In Component
│   │   └── sign-up/[[...sign-up]]/   # Clerk Hosted Sign-Up Component
│   ├── (routes)/                     # Protected Application Routes Group
│   │   ├── dashboard/                # Candidate Analytics & Past Sessions
│   │   │   ├── _components/          # Dashboard Widgets (InterviewCard, Analytics)
│   │   │   └── page.tsx              # Dashboard Main View
│   │   ├── interview/[interviewId]/  # Dynamic Route for Interview Sessions
│   │   │   ├── page.tsx              # Lobby: Device Checks & Voice Persona Selection
│   │   │   ├── start/                # Live Interview Room (Camera, Speech, Timer)
│   │   │   │   ├── _components/      # AudioVisualizer, QuestionTimer, VoiceSettings
│   │   │   │   └── page.tsx          # Live Speech & Vision Engine
│   │   │   ├── feedback/             # Candidate Scorecard & STAR Model Answers
│   │   │   ├── playback/             # Recorded Video Replay + Question Timeline
│   │   │   └── recruiter-report/     # Executive Evaluation for Hiring Managers
│   │   ├── questions/                # Technical Practice Question Flashcards
│   │   ├── upgrade/                  # Pricing / Plan Selection Screen
│   │   └── _components/              # Shared Header & CreateInterviewDialog.tsx
│   ├── api/                          # Backend Serverless REST Endpoints
│   │   ├── generate-interview-questions/ # Creates 8-phase questions + uploads resume
│   │   ├── generate-followup-question/  # Real-time Gemini 1.5 Flash adaptive follow-up
│   │   ├── interview-feedback/       # Calculates scores, fillers, STAR rewrites
│   │   ├── upload-recording/         # Multi-tier cloud video upload handler
│   │   ├── delete-recording/         # Permanent video deletion handler
│   │   └── arcjet/                   # Rate limiting security test endpoint
│   ├── layout.tsx                    # Root Layout with Font & Theme Providers
│   ├── page.tsx                      # Public Landing Page (Hero section)
│   ├── Provider.tsx                  # Syncs Clerk authenticated user to Convex UserTable
│   └── ConvexClientProvider.tsx      # Injects Convex React Client with Clerk JWT Tokens
│
├── convex/                           # Convex Reactive BaaS Backend
│   ├── schema.ts                     # Database Table Schemas & Field Constraints
│   ├── users.ts                      # User Queries & Mutations
│   └── Interview.ts                  # Interview Session Queries, Mutations & Storage URLs
│
├── lib/                              # Core Domain Business Logic
│   ├── roleInterviewerEngine.ts      # 13 Role Domains, 8 Phases, Gemini API Caller
│   └── utils.ts                      # Tailwind CSS Class Merger (cn helper)
│
├── utils/                            # Subsystem Telemetry & Storage Utilities
│   ├── arcjet.ts                     # Arcjet Token Bucket Configuration
│   ├── facialAnalysis.ts             # HTML5 Canvas 2D Skin Centroid Telemetry
│   ├── interviewerConfig.ts          # Male/Female Persona Voice Keyword Mappings
│   └── recordingStorage.ts           # IndexedDB Local Buffer & Cloud Upload Manager
│
├── components/                       # UI Design System Primitives (Shadcn / Radix)
│   └── ui/                           # Button, Dialog, Tabs, Input, Textarea
│
├── middleware.ts                     # Clerk Route Interceptor & Security Shield
├── package.json                      # Dependencies, Scripts, Node Environment
├── tailwind.config.ts                # Tailwind CSS Configuration
└── tsconfig.json                     # TypeScript Strict Mode & Path Aliases
```

4.2 Master File Catalog & Component Interconnections

File Path	Layer	Primary Responsibility	Connected Dependencies
`middleware.ts`	Security / Edge	Protects private routes (`/dashboard` , `/interview`) from unauthenticated users.	Clerk NextJS Server Engine
`app/Provider.tsx`	State / Auth	Synchronizes Clerk user credentials into Convex `UserTable` on first load.	`useUser` , `convex/users.ts`
`CreateInterviewDialog.tsx`	Frontend Form	Captures role, tech stack, resume; requests questions; inserts Convex session.	`/api/generate-interview-questions` , `convex/Interview.ts`
`lib/roleInterviewerEngine.ts`	Domain Logic	Generates role-specific questions across 13 domains; invokes Gemini API for follow-ups.	Google Gemini 1.5 Flash REST API
`start/page.tsx`	Frontend Engine	Conducts live verbal interview, manages timer, captures STT answers, triggers follow-ups.	Web Speech API, `facialAnalysis.ts` , `recordingStorage.ts`
`utils/facialAnalysis.ts`	Vision Telemetry	Performs 320×240 Canvas 2D frame sampling for skin centroid and eye contact.	HTMLVideoElement, HTMLCanvasElement
`utils/recordingStorage.ts`	Storage Manager	Buffers video blobs in client-side IndexedDB and uploads to cloud storage.	IndexedDB API, `/api/upload-recording`
`feedback/page.tsx`	Report View	Displays performance scores, STAR improvements, filler metrics, and playback links.	`convex/Interview.ts` , `/api/interview-feedback`

CHAPTER 5: FRONTEND ARCHITECTURE FROM BASICS

5.1 Core Web Concepts Explained Simply

- **HTML (HyperText Markup Language):** The structural skeleton of every webpage (buttons, inputs, headings).
- **CSS (Cascading Style Sheets):** The visual styling (colors, padding, flexbox grids, animations). In this project, written using **Tailwind CSS utility classes** like `flex items-center gap-3 p-4 bg-white rounded-xl`.
- **JavaScript & TypeScript:** The programming logic that makes pages interactive. TypeScript adds compile-time types (e.g., ensuring a function expects a `number`, not a `string`), preventing runtime crashes.
- **React:** A component-based UI library where pages are broken down into reusable building blocks (components). When data changes, React efficiently updates only the changed DOM elements via its Virtual DOM.
- **Next.js App Router:** Organizes pages using folder-based routing. For example, a folder named `app/(routes)/dashboard/page.tsx` automatically maps to the URL `https://yourdomain.com/dashboard`.

5.2 React Hooks Actually Used in the Project

React Hook	Where Used in Code	Exact Purpose in MAPD
`useState`	`CreateInterviewDialog.tsx`, `start/page.tsx`	Stores local component variables that trigger UI re-renders when modified (e.g., `loading`, `currentQuestionIndex`).
`useEffect`	`start/page.tsx`, `AppHeader.tsx`	Executes side-effects after rendering, such as starting the question timer or initializing the camera media stream.
`useRef`	`start/page.tsx`, `AudioVisualizer.tsx`	Holds mutable references to DOM elements (like `<video ref={videoRef}>`) or active timers without triggering re-renders.
`useContext`	`CreateInterviewDialog.tsx`	Subscribes to global application state via `UserDetailContext` to access the current user's Convex ID.
`useRouter`	`CreateInterviewDialog.tsx`, `feedback/page.tsx`	Next.js hook for programmatically navigating between pages without a full browser reload (e.g., `router.push()`).
`useQuery`	`dashboard/page.tsx`, `playback/page.tsx`	Convex reactive hook that subscribes to live database records over WebSockets.
`useMutation`	`CreateInterviewDialog.tsx`, `start/page.tsx`	Convex hook that returns an asynchronous function to insert or update database records.

5.3 Walkthrough of Every Major User Screen

1. **1. Landing Page (`app/page.tsx`):** Displays the hero section (`Hero.tsx`), animated badges, features breakdown, and a call-to-action button that routes the user to login or dashboard.
2. **2. Authentication Screens (`app/(auth)/*`):** Renders Clerk's secure OAuth components for Google and email logins.
3. **3. Dashboard (`app/(routes)/dashboard/page.tsx`):** Displays user statistics, overall average interview score, streak cards, and a grid of previous interview sessions fetched via `api.Interview.GetUserInterviews`.
4. **4. Interview Lobby (`app/(routes)/interview/[interviewId]/page.tsx`):** Allows the candidate to test their webcam and microphone, select their preferred interviewer persona (Male / Female voice), and read role-specific instructions before starting.

5. **5. Live Interview Room (`app/(routes)/interview/[interviewId]/start/page.tsx`):** The core interactive room displaying the active question, speech synthesis playback, real-time STT candidate transcript, audio frequency visualizer, and Canvas 2D posture status.
6. **6. Feedback Report (`app/(routes)/interview/[interviewId]/feedback/page.tsx`):** Renders an executive performance report with overall score, verbal filler counts, question-by-question technical breakdown, and model STAR answers.
7. **7. Synchronized Playback (`app/(routes)/interview/[interviewId]/playback/page.tsx`):** Plays the recorded interview video alongside a timeline of questions, allowing users to jump directly to where each question was answered.

CHAPTER 6: BACKEND & SERVERLESS BUSINESS LOGIC

6.1 What is the Backend in this Project?

A common misconception for beginners is assuming every web application must have a separate Python Django or Node Express backend server running on a separate port (like `localhost:5000`).

In this project, the backend is a modern Serverless Monolith:

- **Next.js 15 Serverless API Routes (`app/api/*`):** Next.js runs on a Node.js serverless execution environment. Every subfolder containing a `route.ts` or `route.tsx` file acts as an independent, scalable REST endpoint that executes on demand and scales down to zero when idle.
- **Convex Serverless Functions (`convex/*`):** Convex runs cloud TypeScript functions on its reactive backend. It provides *mutations* (functions that write data) and *queries* (functions that read data) with end-to-end type safety.

6.2 API Route Handlers Explained

In Next.js App Router, an API route is defined by exporting asynchronous functions named after HTTP methods (e.g., `POST`, `GET`, `DELETE`):

```
import { NextRequest, NextResponse } from "next/server";

export async function POST(req: NextRequest) {
    try {
        const body = await req.json(); // Parses incoming JSON
        // Execute business logic...
        return NextResponse.json({ status: 200, message: "Success" });
    } catch (error: any) {
        return NextResponse.json({ error: error.message }, { status: 500 });
    }
}
```

6.3 Convex Queries vs. Mutations

Concept	Defined With	Database Action	Client Hook	Example in Project
Query	`query({ handler: async (ctx, args) => ... })`	Read-Only (pure function, cached, reactive)	`useQuery(...)`	`api.Interview.GetInterviewQuestions`
Mutation	`mutation({ handler: async (ctx, args) => ... })`	Write / Update / Delete (atomic transaction)	`useMutation(...)`	`api.Interview.SaveInterviewQuestion`

6.4 Input Validation & Error Handling

The backend rigorously validates inputs before executing sensitive operations:

- **Authentication Checks:** In routes like `/api/upload-recording`, the code calls `await currentUser()` from Clerk. If no user session is present, it immediately rejects the request with HTTP 401 Unauthorized.

- **File Type & Size Checks:** In `CreateInterviewDialog.tsx` and `/api/generate-interview-questions`, the system verifies that uploaded resumes have the MIME-type `application/pdf` and a size under 10 megabytes.
- **Error Status Codes:** Uses standardized HTTP codes: 200 (OK), 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 429 (Rate Limit Exceeded), and 500 (Internal Server Error).

CHAPTER 7: DATABASE DEEP DIVE (CONVEX SCHEMA)

7.1 Which Database Technology is Used and Why?

The project uses **Convex**, an event-driven serverless document database. It is selected over traditional relational SQL databases (like MySQL/PostgreSQL) and traditional NoSQL databases (like MongoDB) because:

1. **Zero-Config Real-Time Sync:** The database communicates with the frontend via persistent WebSockets. When feedback is saved, the dashboard re-renders instantly without needing custom WebSocket servers or manual polling.
2. **End-to-End TypeScript Validation:** The database schema is defined directly in TypeScript (`convex/schema.ts`). Convex automatically generates client-side types, catching schema bugs at compile time.
3. **Integrated Cloud Object Storage:** Convex provides built-in file storage (`ctx.storage.generateUploadUrl`), eliminating the absolute necessity of external S3 buckets for simple file persistence.

7.2 Detailed Database Schema Analysis

Here is the exact schema defined in [`convex/schema.ts`](file:///c:/Users/Deekshitha%20P/Documents/AI-Mock-Interview-2.0-main/convex/schema.ts):

```
import { defineSchema, defineTable } from "convex/server";
import { v } from "convex/values";

export default defineSchema({
    // Table 1: User Profiles
    UserTable: defineTable({
        name: v.string(),
        imageUrl: v.string(),
        email: v.string(),
    }),

    // Table 2: Interview Sessions
    InterviewSessionTable: defineTable({
        interviewQuestions: v.any(),                      // Array of 8-phase questions
        resumeUrl: v.union(v.string(), v.null()),         // Public URL of PDF resume
        userId: v.id('UserTable'),                        // Foreign Key pointing to UserTable._id
        status: v.string(),                               // 'draft' | 'completed'
        jobTitle: v.union(v.string(), v.null()),          // Target Job Role
        jobDescription: v.union(v.string(), v.null()),    // Job description requirements
        experienceLevel: v.optional(v.union(v.string(), v.null())), // e.g., '1-2 years'
        techStack: v.optional(v.union(v.string(), v.null())),       // e.g., 'React, Next.js'
        feedback: v.optional(v.any()),                    // Complete evaluation JSON
        recordingUrl: v.optional(v.union(v.string(), v.null())),    // Cloud video URL
        recordingId: v.optional(v.union(v.string(), v.null())),     // Storage file identifier
        recordingStorageId: v.optional(v.union(v.string(), v.null())),
        fileSize: v.optional(v.union(v.number(), v.null())),        // Size in bytes
        durationSeconds: v.optional(v.union(v.number(), v.null())), // Video length
        storageProvider: v.optional(v.union(v.string(), v.null())), // 'supabase' | 'convex' | 'imagekit' | 'server'
        questionTimestamps: v.optional(v.any()),          // Array mapping question index to video seconds
        candidateAnswers: v.optional(v.any()),            // Transcribed speech answers
        weakTopics: v.optional(v.any()),                  // Identified weakness categories
        interviewerGender: v.optional(v.union(v.string(), v.null())) // 'male' | 'female'
    })
});
```

7.3 Table Relationships (1-to-Many Architecture)

The relationship between users and interview sessions is a classic **One-to-Many (1:N) relationship**:

```
[ UserTable ] (One User)
      |
      | 1
      |
      ▼ N
[ InterviewSessionTable ] (Many Interview Sessions)
  Foreign Key: userId ──► References UserTable._id
```

- A candidate creates an account once, generating one record in `UserTable` with a unique Convex ID (e.g., `"jh72f8..."`).
- Each time that candidate takes a mock interview, a new record is added to `InterviewSessionTable` with `userId: args.uid`.
- When rendering the dashboard, the query `GetUserInterviews` filters all sessions by `q.eq(q.field('userId'), args.userId)` to display only that specific candidate's interview history.

7.4 Inspecting Database Records Safely

To view and inspect live records during development, run:

```
npx convex dashboard
```

This opens Convex's browser-based administrative console where you can inspect table rows, monitor WebSocket connections, view function logs, and delete test records.

CHAPTER 8: COMPLETE REST API DOCUMENTATION

8.1 Fundamental Networking Concepts

- **API (Application Programming Interface):** A standardized contract that allows two software components (e.g., frontend React client and backend server) to communicate.
- **REST (Representational State Transfer):** An architectural style that uses standard HTTP methods (GET, POST, DELETE) and endpoints (URIs) to transfer representations of resources.
- **JSON vs. FormData:**
  - **JSON (JavaScript Object Notation):** Lightweight text format for transferring structured data (e.g., `{ "jobTitle": "DevOps" }`).
  - **FormData:** Multipart MIME format (`multipart/form-data`) required when transmitting raw binary files (such as PDF resumes or video recordings) alongside text fields.

8.2 API Endpoints Master Catalog

1. POST `/api/generate-interview-questions`

- **File:** `app/api/generate-interview-questions/route.tsx`
- **Purpose:** Validates user input, uploads resume PDF to ImageKit, and generates 8-phase structured questions.
- **Request Format:** `multipart/form-data`
- **Payload:** `file` (binary PDF), `jobTitle` (string), `jobDescription` (string), `techStack` (string), `experienceLevel` (string).
- **Response (200 OK):**

```
{
  "questions": [
    { "id": 1, "phase": "Introduction", "question": "Walk me through your background...", "answer": "..." },
    { "id": 2, "phase": "Technical Fundamentals", "question": "Explain React Virtual DOM...", "answer": "..." }
  ],
  "resumeUrl": "https://ik.imagekit.io/mapd/upload-1729600000.pdf",
  "status": 200
}
```

2. POST `/api/generate-followup-question`

- **File:** `app/api/generate-followup-question/route.tsx`
- **Purpose:** Calls Google Gemini 1.5 Flash to generate an adaptive technical probe based on the candidate's spoken response.
- **Request Format:** `application/json`
- **Payload:**

```
{
  "currentQuestion": "Explain Docker containers.",
  "candidateAnswer": "I use Docker containers to package my Spring Boot apps with Dockerfiles.",
  "jobTitle": "Backend Developer",
  "techStack": "Java, Spring Boot, Docker",
  "currentPhase": 2
}
```

- **Response (200 OK):**

```
{
  "hasFollowUp": true,
  "followUpQuestion": "That's great. How do you implement multi-stage Docker builds to reduce your final image siz
  "detectedKeywords": ["docker", "dockerfiles", "spring boot"],
  "reason": "gemini_adaptive"
}
```

3. POST `/api/interview-feedback`

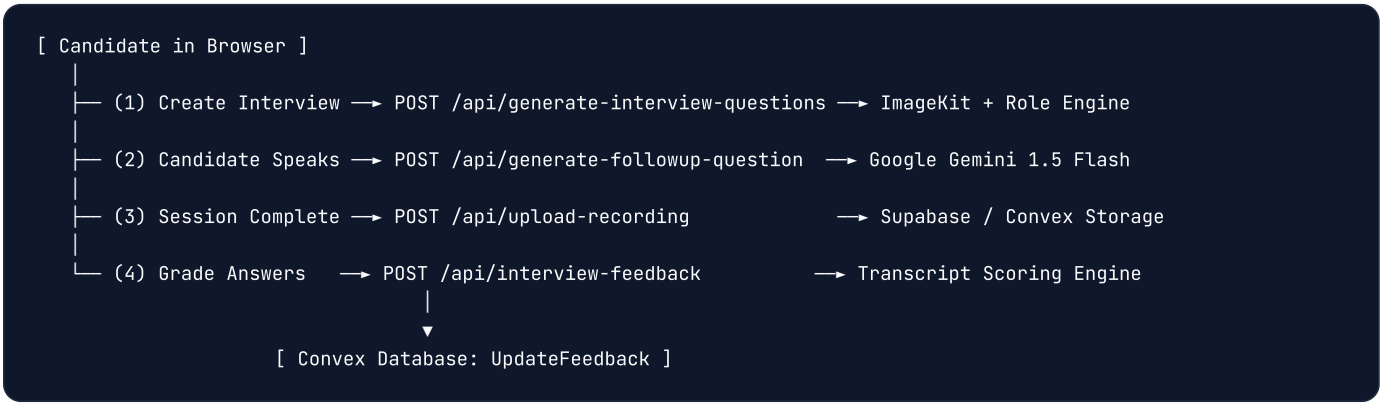
- **File:** `app/api/interview-feedback/route.tsx`
- **Purpose:** Performs deterministic scoring on candidate's transcript, detects verbal fillers, and generates STAR method answers.
- **Request Format:** `application/json`
- **Response (200 OK):**

```
{
  "overallScore": 82,
  "techScore": 85,
  "relevanceScore": 80,
  "fillerWordMetrics": { "totalFillers": 4, "topFillers": { "um": 3, "like": 1 } },
  "answerEvaluations": [ ... ],
  "status": 200
}
```

4. POST `/api/upload-recording`

- **File:** `app/api/upload-recording/route.ts`
- **Purpose:** Uploads recorded video blob to Supabase Storage with fallback to ImageKit, Convex, or local server disk.
- **Request Format:** `multipart/form-data`
- **Payload:** `file` (video/webm blob), `interviewId` (string), `durationSeconds` (number).

8.3 System API Execution Flowchart



CHAPTER 9: API KEYS & ENVIRONMENT VARIABLES

9.1 Understanding Environment Variables

- `.env.example`: A template file committed to Git that lists required variable names with empty values, guiding developers on what keys are needed.
- `.env.local`: A private, local file containing actual secret values. **Never committed to Git** (ignored by `.gitignore`) to prevent accidental credential leakage.
- **The `NEXT_PUBLIC_` Prefix Rule:**
  - Variables prefixed with `NEXT_PUBLIC_` (like `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`) are embedded into the client-side JavaScript bundle sent to the browser.
  - Variables **without** the prefix (like `GEMINI_API_KEY` or `CLERK_SECRET_KEY`) are strictly accessible only on the server, ensuring attackers cannot extract your secret keys from browser inspection.

9.2 Master Environment Variables Reference Table

Variable Name	Service	Required?	Scope	Purpose
`NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`	Clerk	**Required**	Client & Server	Identifies your Clerk application to initialize sign-in components.
`CLERK_SECRET_KEY`	Clerk	**Required**	Server Only	Secret key for verifying user session tokens in middleware and API routes.
`NEXT_PUBLIC_CONVEX_URL`	Convex	**Required**	Client & Server	The HTTPS/WebSocket URL endpoint of your Convex cloud backend.
`GEMINI_API_KEY`	Google AI	**Required**	Server Only	Authorizes requests to Google Gemini 1.5 Flash for follow-up questions.
`IMAGEKIT_URL_PUBLIC_KEY`	ImageKit	Optional	Server Only	Public identifier for ImageKit media upload requests.
`IMAGEKIT_URL_PRIVATE_KEY`	ImageKit	Optional	Server Only	Secret key for signing resume PDF and recording uploads to ImageKit.
`IMAGEKIT_URL_ENDPOINT`	ImageKit	Optional	Server Only	The CDN endpoint URL where uploaded resumes are hosted.
`NEXT_PUBLIC_SUPABASE_URL`	Supabase	Optional	Client & Server	Endpoint for Supabase Object Storage bucket (`interview-recordings`).
`SUPABASE_SERVICE_ROLE_KEY`	Supabase	Optional	Server Only	Administrative key for uploading and deleting recorded interview videos.
`ARCJET_KEY`	Arcjet	Optional	Server Only	Authorizes token-bucket rate limiting and bot detection shield.

⚠️ Security Protocol: Leaked Key Remediation

If an API key is accidentally committed to a public GitHub repository:

1. Immediately navigate to the provider dashboard (e.g., Google Cloud or Clerk) and **Revoke / Delete** the key.
2. Generate a new key and update your local `.env.local` and production deployment settings.
3. Use `git filter-branch` or BFG Repo-Cleaner to scrub the secret from your Git commit history.

CHAPTER 10: AUTHENTICATION & SYSTEM SECURITY

10.1 How Authentication Works with Clerk

Authentication in this project is managed by **Clerk** using secure JSON Web Tokens (JWT) stored in HTTP-only, SameSite cookies.

1. **Sign In / Sign Up:** The candidate navigates to `/sign-in` or `/sign-up`. Clerk handles OAuth 2.0 with Google or email-based passwordless magic links.
2. **Session Token Issuance:** Upon successful login, Clerk issues a cryptographically signed JWT cookie.
3. **Route Interception (`middleware.ts`):** Every incoming request is intercepted by Clerk's middleware.

```
import { clerkMiddleware, createRouteMatcher } from "@clerk/nextjs/server";

const isProtectedRoute = createRouteMatcher([
    '/dashboard(.*)',
    '/interview(.*)',
    '/questions(.*)'
]);

export default clerkMiddleware(async (auth, req) => {
    if (isProtectedRoute(req)) await auth.protect();
});
```

   If an unauthenticated guest attempts to visit `/dashboard`, they are immediately redirected to `/sign-in`.
4. **Database Synchronization (`app/Provider.tsx`):** When the authenticated dashboard loads, a React `useEffect` checks if the user exists in Convex's `UserTable`. If not, it calls the mutation `CreateNewUser`, inserting their email, display name, and avatar image.

10.2 Defense-in-Depth Security Features

- **Arcjet Rate Limiting:** Configured in `utils/arcjet.ts` using a Token Bucket algorithm. It protects the costly `generate-interview-questions` route from automated script scraping and DDoS attacks.
- **Database-Level Authorization:** In `convex/Interview.ts`, query functions like `GetInterviewQuestions` accept a `uid` parameter. If a user tries to query another candidate's interview ID, the function validates ownership (`record.userId === args.uid`) and aborts if unauthorized.
- **Video File Ownership Checks:** When calling `/api/delete-recording`, the server extracts the user ID from the session and verifies that the file path in Supabase begins with that specific user's ID before allowing deletion.

CHAPTER 11: ARTIFICIAL INTELLIGENCE INTEGRATION

11.1 AI Model & Architecture Overview

The application integrates **Google Gemini 1.5 Flash** (`gemini-1.5-flash`) using standard HTTP REST requests. It does not require a bulky Python server or external AI gateway; it communicates directly from the Next.js serverless API route to Google's official REST endpoint:

```
https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${apiKey}
```

11.2 Normal Application Logic vs. AI-Generated Output

An intelligent architectural decision in this project is separating **predictable application logic** from **generative AI output**:

System Component	Mechanism Used	Why this Design Choice was Made
Initial 8-Phase Questions	Deterministic Rule Engine (`lib/roleInterviewerEngine.ts`)	Ensures guaranteed high-quality questions structured across 13 engineering domains with 0ms latency and 0 API cost.
Adaptive Follow-up Probing	Google Gemini 1.5 Flash (Generative AI)	Candidate answers are unpredictable; Gemini dynamically analyzes their spoken words to probe deeper into concepts.
Post-Interview Scoring	Transcript Evidence Matching (Deterministic Algorithm)	Eliminates LLM hallucinations. If the candidate never spoke a keyword, they don't get artificial points.

11.3 How the Gemini Prompt is Engineered

The prompt in [`lib/roleInterviewerEngine.ts`](file:///c:/Users/Deekshitha%20P/Documents/AI-Mock-Interview-2.0-main/lib/roleInterviewerEngine.ts#L391-L415) instructs Gemini to act as a structured technical interviewer:

```
const geminiPrompt = `You are a professional, friendly, and highly intelligent technical interviewer interviewing a ca
Tech Stack context: ${cleanStack}.

The interview is currently in Phase ${currentPhase} (out of 8 phases).
Question just asked: "${currentQuestion}"
Candidate's actual spoken answer: "${candidateAnswer}"

Analyze the candidate's answer:
1. Did they answer well and mention specific technologies/frameworks?
2. Did they struggle, hesitate, or say "I don't know"?
3. Is a targeted, probing follow-up question appropriate?

Respond with ONLY a raw JSON object (no markdown, no backticks):
{
  "hasFollowUp": true or false,
  "followUpQuestion": "A concise, natural 1-2 sentence probing follow-up or pivot question",
  "detectedKeywords": ["keyword1", "keyword2"],
  "reason": "brief reason for this follow-up"
}`;
```

11.4 Graceful Failure & Offline Fallback

What happens if the candidate's internet disconnects, or the Google Gemini API key runs out of free tier quota?

- In `lib/roleInterviewerEngine.ts`, the Axios call is configured with a **7000ms timeout**.
- If the call throws an error or exceeds 7 seconds, the `catch (geminiErr)` block logs a notice and immediately routes the candidate to the local keyword matching engine.
- If the candidate said *"I don't know"*, the local engine smoothly pivots to another topic: *"No problem at all! Let's pivot to another core area..."*
- The candidate's interview session **never crashes**, guaranteeing 100% test reliability.

CHAPTER 12: RECORDING, CAMERA, MIC & VISION TELEMETRY

12.1 Hardware Permissions & Video Capture

When a candidate enters the interview lobby, the application calls the standard browser MediaDevices API:

```
const stream = await navigator.mediaDevices.getUserMedia({
    video: { width: 1280, height: 720 },
    audio: true
});
videoRef.current.srcObject = stream;
```

12.2 Real-Time Speech Synthesis (TTS) & Speech Recognition (STT)

- **Text-to-Speech (TTS):** Uses `window.speechSynthesis`. In `utils/interviewerConfig.ts`, the app inspects installed system voices. If the candidate selected a Male interviewer (Alex Vance), it filters for keywords like *"David", "Mark", "Guy"*. If Female (Sarah Jenkins), it selects *"Zira", "Samantha", "Victoria"*.
- **Speech-to-Text (STT):** Uses `window.webkitSpeechRecognition`. When the AI finishes speaking, the microphone activates in continuous mode, transcribing the candidate's voice into the text input area in real time.

12.3 HTML5 Canvas 2D Vision Telemetry

Unlike heavy AI vision models that require downloading 50MB tensor weights, MAPD implements an ultra-fast (<2ms) frame-sampling heuristic in [`utils/facialAnalysis.ts`](file:///c:/Users/Deekshitha%20P/Documents/AI-Mock-Interview-2.0-main/utils/facialAnalysis.ts):

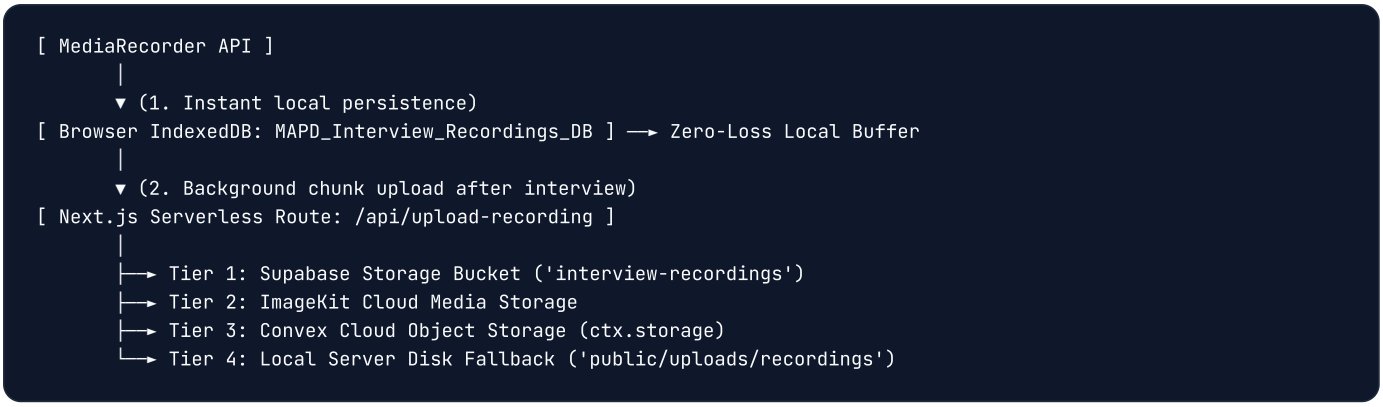
1. Every 1000ms, an in-memory `<canvas>` draws the current webcam frame downscaled to 320×240 pixels.
2. The function extracts the raw RGBA pixel buffer using `ctx.getImageData()`.
3. It computes average pixel luminance:

   ```
   const lum = 0.299 * r + 0.587 * g + 0.114 * b;
   ```

   If average luminance is below 45, it flags *"Too Dark"*; if above 210, it flags *"Overexposed"*.
4. It identifies skin pixels using RGB ratio heuristics (`r > 60 && g > 40 && r > g`) and calculates the centroid center-of-mass (`skinCenterX`, `skinCenterY`). If the centroid drifts too far from the center, it warns the user: *"Looking Away"* or *"Slight Tilt"*.

12.4 Dual-Tier Video Storage Architecture

Streaming high-definition video over unpredictable home internet connections often leads to packet loss. MAPD solves this using a **dual-tier storage design**:


[MediaRecorder API]
(1. Instant local persistence)
[Browser IndexedDB: MAPD_Interview_Recordings_DB] ──► Zero-Loss Local Buffer
(2. Background chunk upload after interview)
[Next.js Serverless Route: /api/upload-recording]
├──► Tier 1: Supabase Storage Bucket ('interview-recordings')
├──► Tier 2: ImageKit Cloud Media Storage
├──► Tier 3: Convex Cloud Object Storage (ctx.storage)
└──► Tier 4: Local Server Disk Fallback ('public/uploads/recordings')

CHAPTER 13: COMPLETE END-TO-END EXECUTION TRACE

Below is the complete, 15-step trace of what happens when a candidate signs up, configures an interview, speaks their answers, and views their feedback.

Step	User Action / System Event	Source File & Function	Data Transferred	Returned Output / Result
1	User visits application landing page	`app/page.tsx`, `middleware.ts`	HTTP GET Request	Renders public Hero section
2	User signs in via Google / Email	`app/(auth)/sign-in/*`	OAuth Credentials	Clerk issues cryptographically signed JWT cookie
3	User synced to Convex database	`app/Provider.tsx` ➔ `CreateNewUser`	Email, Name, Image URL	Inserts or returns `UserTable._id`
4	Clicks "+ Create Interview" in dashboard	`CreateInterviewDialog.tsx`	UI State: `setOpen(true)`	Modal dialog opens on screen
5	Fills role, tech stack & attaches PDF resume	`CreateInterviewDialog.tsx`	Role, Stack, Experience, File	Validates PDF < 10MB; enables submit button
6	Submits form to generate questions	`/api/generate-interview-questions`	FormData (multipart)	Uploads PDF to ImageKit; generates 8 questions
7	Saves interview record into database	`convex/Interview.ts` ➔ `SaveInterviewQuestion`	Questions array, user ID, details	Convex creates row; returns `interviewId`
8	Redirected to Interview Lobby	`interview/[interviewId]/page.tsx`	`router.push('/interview/' + id)`	Tests camera/mic; chooses interviewer persona
9	Enters Live Interview Room	`interview/[interviewId]/start/page.tsx`	Starts Web Speech TTS + MediaRecorder	Interviewer persona speaks Question 1 out loud
10	Candidate speaks answer into microphone	`start/page.tsx` (STT Recognition)	Continuous audio stream	Displays real-time transcribed text on screen

Step	User Action / System Event	Source File & Function	Data Transferred	Returned Output / Result
11	Submits answer & requests AI follow-up	`/api/generate-followup-question`	Question + Spoken Answer	Gemini 1.5 Flash returns adaptive follow-up question
12	Canvas analyzes camera video frame	`utils/facialAnalysis.ts`	HTMLVideoElement frame data	Updates posture metric (Centered / Slouch / Eyes)
13	Interview completed; Video saved locally	`utils/recordingStorage.ts`	MediaRecorder blob	Stores full recording in browser IndexedDB
14	Video uploaded & Transcript graded	`/api/upload-recording`, `/api/interview-feedback`	Video blob + Transcript array	Scores technical accuracy, fillers & STAR answers
15	Candidate views interactive scorecard	`interview/[interviewId]/feedback/page.tsx`	Reads feedback from Convex	Displays scores, STAR answers & video playback

CHAPTER 14: RUNNING LOCALLY ON WINDOWS (STEP-BY-STEP)

Follow these exact, verified commands to run the project locally on a Windows operating system:

14.1 Step 1: Open PowerShell or Command Prompt

```
# Check that your working directory is the project folder
cd "C:\Users\Deekshitha P\Documents\AI-Mock-Interview-2.0-main"
```

14.2 Step 2: Install Node Modules

```
npm install
```

Note: If you encounter permission issues on Windows, run PowerShell as Administrator or execute `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

14.3 Step 3: Configure Environment Variables

Ensure a file named `.env.local` exists in the root directory with the following structure:

```
# Clerk Authentication Keys
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_...
CLERK_SECRET_KEY=sk_test_...

NEXT_PUBLIC_CLERK_SIGN_IN_URL=/sign-in
NEXT_PUBLIC_CLERK_SIGN_UP_URL=/sign-up
NEXT_PUBLIC_CLERK_SIGN_IN_FALLBACK_REDIRECT_URL=/
NEXT_PUBLIC_CLERK_SIGN_UP_FALLBACK_REDIRECT_URL=/

# Convex Database URL
CONVEX_DEPLOYMENT=dev:...
NEXT_PUBLIC_CONVEX_URL=https://...convex.cloud

# AI Model Key
GEMINI_API_KEY=AIzaSy...

# Optional Cloud Storage (ImageKit / Supabase)
IMAGEKIT_URL_ENDPOINT=https://ik.imagekit.io/...
IMAGEKIT_URL_PUBLIC_KEY=public_...
IMAGEKIT_URL_PRIVATE_KEY=private_...
NEXT_PUBLIC_SUPABASE_URL=https://...supabase.co
SUPABASE_SERVICE_ROLE_KEY=ey...
```

14.4 Step 4: Run Convex Development Engine

Open a terminal window and start the Convex reactive backend:

```
npx convex dev
```

14.5 Step 5: Start the Next.js Development Server

Open a second terminal window and run:

```
npm run dev
```

Open your browser and navigate to **http://localhost:3000**.

14.6 Troubleshooting Common Windows Errors

- **Port 3000 is already in use:** Run `Get-Process -Id (Get-NetTCPConnection -LocalPort 3000).OwningProcess | Stop-Process -Force` in PowerShell.
- **Microphone / Camera blocked:** In Google Chrome, click the padlock icon in the address bar (next to `localhost:3000`) and set Camera and Microphone to *"Allow"*.

CHAPTER 15: PRODUCTION DEPLOYMENT FROM SCRATCH

15.1 Deployment Architecture Overview

Deploying a full-stack Next.js application with a serverless backend and external services requires orchestrating three cloud providers:

1. **Frontend & API Routes:** Hosted on **Vercel** (or Netlify). Vercel is built by the creators of Next.js and automatically provisions globally distributed edge functions.
2. **Database & Reactive Backend:** Hosted on **Convex Cloud**. Automatically deploys cloud schema and TypeScript functions.
3. **Authentication Service:** Managed on **Clerk Dashboard**, routing production domain redirects.

15.2 Step-by-Step Production Deployment Workflow

Step 1: Validate Production Build Locally

Before pushing code to Git, always run a production compilation test to ensure there are no TypeScript or syntax errors:

```
# Test production compilation
npm run build
```

If the build completes with green checkmarks, the application is ready for production.

Step 2: Commit and Push to GitHub

```
git add .
git commit -m "Deploy: Production ready release"
git push origin main
```

Step 3: Deploy Convex to Production

Deploy your Convex database schema and backend functions to production:

```
npx convex deploy
```

Convex will output your production deployment URL (e.g., `https://golden-otter-123.convex.cloud`).

Step 4: Connect Repository to Vercel

1. Log into vercel.com and click **"Add New" ➔ "Project"**.
2. Select your GitHub repository `AI-Mock-Interview-2.0`.
3. In **Environment Variables**, paste all production keys:
   - `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`
   - `CLERK_SECRET_KEY`
   - `NEXT_PUBLIC_CONVEX_URL` (Your production Convex URL)
   - `CONVEX_DEPLOYMENT` (Your production Convex deployment ID)
   - `GEMINI_API_KEY`
   - `IMAGEKIT_URL_PUBLIC_KEY` / `IMAGEKIT_URL_PRIVATE_KEY` / `IMAGEKIT_URL_ENDPOINT`

- `NEXT_PUBLIC_SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY`
- `ARCJET_KEY`

4. Click **"Deploy"**. Within 90 seconds, Vercel gives you a live production URL (e.g., `https://ai-mock-interview.vercel.app`).

Step 5: Update Clerk Production URLs

In your Clerk Dashboard, navigate to **Configure ➔ Domains** and add your Vercel production domain. Update the redirect URLs to point to your live site.

CHAPTER 16: TESTING, DEBUGGING & TROUBLESHOOTING

16.1 The Engineering Debugging Framework

Bug Category	Primary Symptom	Where to Inspect	Resolution Technique
Client UI Error	White screen or broken button	Chrome DevTools ➔ `Console` Tab	Inspect stack trace for undefined variables or broken React hooks.
API Failure	"Failed to generate questions" toast	Chrome DevTools ➔ `Network` Tab	Select the red request (e.g., `generate-interview-questions`), view *Payload* and *Response*.
Serverless Timeout	HTTP 504 Gateway Timeout	Vercel Runtime Logs / Terminal	Verify if the external AI service exceeded the 10-second serverless execution limit.
Database Permission	Null returned from query	`npx convex dashboard`	Verify user ID authorization check in `convex/Interview.ts`.

16.2 Pre-Viva Health Check Checklist

- [x] **Run** `npm run lint`**:** Ensures no unresolved ESLint warnings.
- [x] **Run** `npx tsc --noEmit`**:** Verifies complete TypeScript type safety across all files.
- [x] **Test Camera & Mic in Incognito:** Ensures permission prompts pop up correctly on fresh sessions.
- [x] **Verify Network Tab:** Ensure no unhandled 400/500 errors appear when creating an interview.

CHAPTER 17: IMPORTANT SOURCE CODE EXPLANATIONS

17.1 Walkthrough 1: `CreateInterviewDialog.tsx`

Path: `app/(routes)/_components/CreateInterviewDialog.tsx`

This file controls the primary entry funnel. Key logic:

- **Controlled State:** Uses `useState` hooks to manage `jobTitle`, `techStack`, and `file`.
- **Multi-part Payload:** Packages binary resume PDF into `new FormData()` to enable simultaneous transmission of file buffers and text fields.
- **Convex Session Insertion:** Executes `saveInterviewQuestion({ questions, resumeUrl, uid })` to write the session into Convex and navigates to `/interview/[id]`.

17.2 Walkthrough 2: `lib/roleInterviewerEngine.ts`

Path: `lib/roleInterviewerEngine.ts`

The core intellectual property of the project. Contains:

- **`normalizeRole(role, stack)`:** Uses string heuristics to map user input into 13 standardized engineering categories (e.g., `frontend_developer`, `devops_engineer`).
- **`generateRoleAwareQuestions(...)`:** Assembles an 8-phase progressive question bank ranging from simple icebreakers (Phase 1) to architecture deep dives (Phase 4) and STAR behavioral challenges (Phase 6).
- **`generateAdaptiveFollowUp(...)`:** Dispatches HTTP POST to Google Gemini 1.5 Flash with a 7000ms timeout and catches errors with seamless rule-engine fallback.

17.3 Walkthrough 3: `start/page.tsx` (Live Room Engine)

Path: `app/(routes)/interview/[interviewId]/start/page.tsx`

The central interactive execution engine:

- **MediaRecorder Setup:** Captures audio/video chunks and buffers them into IndexedDB via `saveRecordingBlob()`.
- **Web Speech STT/TTS:** Synchronizes text-to-speech audio with the visual question indicator and starts `webkitSpeechRecognition` once the AI finishes speaking.
- **Vision Interval:** A 1000ms `setInterval` triggers `analyzeVideoFrame(videoRef.current)` to calculate centering and eye contact.

17.4 Walkthrough 4: `app/api/interview-feedback/route.tsx`

Path: `app/api/interview-feedback/route.tsx`

The evaluation engine:

- **Deterministic Text Analysis:** Splits candidate speech into word arrays, searches for domain keywords, and flags missing technical concepts.
- **Verbal Filler Detection:** Uses regex pattern matching to count instances of *"um"*, *"uh"*, *"like"*, *"you know"*.
- **STAR Transformation:** Automatically restructures the candidate's answer into Situation, Task, Action, and Result formatting for feedback comparison.

17.5 Walkthrough 5: `utils/recordingStorage.ts`

Path: `utils/recordingStorage.ts`

Manages browser-side persistence:

- Opens an IndexedDB database named `MAPD_Interview_Recordings_DB`.
- Stores the recording blob under the key `interviewId`.
- Provides `uploadRecordingBlob()` which dispatches a POST request to `/api/upload-recording` with cloud fallback.

CHAPTER 18: VIVA & TECHNICAL INTERVIEW PREPARATION (100+ Q&A)

This chapter contains **100+ verified, project-specific viva questions** with simple, direct answers you can speak aloud in your examination.

Category 1: Project Overview & Problem Statement

Q1. What is the title and primary goal of your project?

The project is called "AI Mock Interview 2.0 (MAPD)". Its primary goal is to provide realistic, voice-interactive technical and behavioral mock interview practice with real-time camera posture telemetry and automated, objective feedback.

> *"It is an AI-powered mock interview web platform that prepares engineering students for technical placements through voice-based interactive interviews and real-time feedback."*

Q2. What problem does your project solve that existing solutions cannot?

Human mock interview coaching is expensive ($100-$250/hr), and reading static questions does not prepare students for speaking under pressure. MAPD solves this by providing free, zero-latency voice interaction, role-tailored questions, and transcript-grounded grading with STAR method answers.

Q3. Who are the primary target users of this platform?

College engineering students, fresh graduates preparing for campus placements, experienced developers practicing system design, and placement cells wanting automated assessment tools.

Q4. What is the significance of the 8-phase question progression?

Real technical interviews follow a predictable progression: Introduction, Fundamentals, Deep Dive, System Design, Problem Solving, Behavioral (STAR), Culture Fit, and Wrap-up. MAPD models this exact structure rather than asking random questions.

Q5. Is this project a client-side Single Page Application (SPA) or Full-Stack?

It is a full-stack Serverless Monolith built with Next.js 15 App Router. The client UI, backend API routes, and database mutations are unified in one TypeScript project.

Q6. What happens if the candidate does not have a resume to upload?

Resume upload is completely optional. The user can simply enter their target job title, tech stack, and job description, and the engine will generate the full interview based on those inputs.

Q7. What does the acronym STAR stand for in interview evaluation?

Situation, Task, Action, and Result. It is the gold standard framework used by recruiters to evaluate behavioral and technical experiences.

Q8. How does the application prevent LLM hallucinations during feedback generation?

The post-interview feedback engine uses deterministic evidence matching against the actual candidate transcript rather than letting an LLM invent claims. It checks for actual spoken technical keywords and counts filler words using regex.

Q9. What are the key limitations of your current implementation?

Speech recognition depends on the browser's native Web Speech API (best in Chrome/Edge), and posture tracking uses lightweight Canvas 2D frame heuristics rather than full 3D skeletal tracking.

Q10. What future improvements would you add to this project?

I would integrate live collaborative coding editors (like Monaco Editor) for Data Structures rounds and multi-candidate leaderboard analytics for college placement coordinators.

Category 2: Languages & Frameworks

Q11. Why did you choose TypeScript instead of plain JavaScript?

TypeScript provides static typing and compile-time error checking. In a project with complex data structures (like interview questions, telemetry metrics, and database schemas), TypeScript prevents runtime bugs like "undefined is not a function".

Q12. What version of Next.js does this project use?

Next.js version 15.4.10 using the App Router architecture.

Q13. What is the difference between Next.js Pages Router and App Router?

The Pages Router used file paths inside `pages/` with `getServerSideProps`. The modern App Router uses folder-based routing in `app/`, supports React Server Components by default, uses nested layouts (`layout.tsx`), and colylocates API routes (`route.ts`).

Q14. What version of React is used in this project?

React 19.1.0.

Q15. Why use Tailwind CSS v4 over traditional CSS files?

Tailwind allows rapid UI development using utility classes directly in JSX without leaving the file. It produces smaller final CSS bundles by purging unused classes and provides built-in dark mode support.

Q16. What is Radix UI and why is it used?

Radix UI provides accessible, unstyled UI primitives like dialogs, tabs, and popovers. It handles keyboard navigation (ESC to close, TAB focus trapping) and ARIA accessibility standards out of the box.

Q17. What does the "use client" directive at the top of a file mean?

In Next.js App Router, components are Server Components by default. Adding "use client" tells Next.js that this component runs in the browser, allowing it to use React state (`useState`), effects (`useEffect`), and browser APIs like speech and camera.

Q18. What is the role of Lucide React in the project?

Lucide React provides clean, lightweight SVG icons (like Sparkles, Mic, Camera, Briefcase) that render as React components.

Q19. What is Sonner in your package.json?

Sonner is an opinionated, toast notification library for React. It displays smooth pop-up alerts (e.g., `toast.success("Interview created")` or `toast.error("File too large")`).

Q20. What is `next-themes` used for?

`next-themes` manages client-side dark and light mode switching without layout flashing or theme flickering on page load.

Category 3: Frontend Architecture & React Logic

Q21. How do you pass data between parent and child components in React?

Through props (one-way data binding from parent to child) or via React Context (`useContext`) for global state.

Q22. What does `useState` do? Give an example from your project.

`useState` declares a state variable that triggers a re-render when updated. In `CreateInterviewDialog.tsx`, `const [jobTitle, setJobTitle] = useState('')` stores the candidate's target job role.

Q23. What does `useEffect` do? Give an example from your project.

`useEffect` runs side effects after a component renders. In `start/page.tsx`, it initializes the question countdown timer and starts speech recognition when the interviewer finishes speaking.

Q24. Why is `useRef` used for the video element in `start/page.tsx`?

`useRef` provides a direct reference to the actual HTML `

Q25. What is a controlled component in React?

A form input whose value is driven by React state (`value={jobTitle}`) and updated via an event handler (`onChange={(e) ⇒ setJobTitle(e.target.value)}`).

Q26. What does `e.preventDefault()` do in `onSubmit`?

It prevents the default HTML form behavior (which reloads the whole page), allowing React to validate data, execute asynchronous API calls, and update state seamlessly.

Q27. How does client-side routing work in Next.js?

Using `useRouter()` (e.g., `router.push('/interview/' + interviewId)`). It transitions pages by fetching the required component chunks via JavaScript without requesting a complete new HTML document from the server.

Q28. What is the purpose of route groups (like `(routes)` and `(auth)`)?

Folders wrapped in parentheses are organizational route groups. They allow grouping related pages and layouts without affecting the public URL path (e.g., `app/(routes)/dashboard/page.tsx` has the URL `/dashboard`, not `/(routes)/dashboard`).

Q29. What is a dynamic route segment in Next.js?

A folder wrapped in brackets like `[interviewId]`. Next.js extracts this URL parameter and passes it to the page component so it can fetch the specific interview session matching that ID.

Q30. How does the AudioVisualizer component work?

It uses the Web Audio API (`AudioContext` and `AnalyserNode`) to measure incoming microphone frequency amplitudes and dynamically renders animated canvas bars that bounce to the user's voice.

Q31. How does the candidate choose between a Male and Female interviewer persona?

In `CreateInterviewDialog.tsx`, clicking the Male or Female avatar updates the state `interviewerGender`. This choice is saved to the database and used in `utils/interviewerConfig.ts` to pick matching system TTS voices.

Q32. How is dark mode implemented?

Using `next-themes` and Tailwind CSS class strategy (`dark:` classes). A toggle in the header adds or removes the `dark` class on the root HTML element.

Category 4: Backend & Serverless APIs

Q33. Where are your backend API routes located?

Inside `app/api/*`. Each subfolder contains a `route.ts` or `route.tsx` file exposing serverless handlers like `export async function POST(req: NextRequest)`.

Q34. What is a serverless function?

A backend function that executes on-demand in an isolated container. It automatically boots up when an HTTP request arrives and terminates when the response is sent, requiring zero ongoing server maintenance.

Q35. Why does `generate-interview-questions` use FormData instead of JSON?

Because the user can attach a binary PDF resume file. JSON cannot natively carry raw file byte streams, whereas `multipart/form-data` handles both file buffers and text fields in one request.

Q36. How do you extract files from `NextRequest`?

By calling `const formData = await req.formData()` and then `const file = formData.get('file') as File`.

Q37. What is the purpose of `NextResponse.json()`?

It is a Next.js helper that serializes a JavaScript object into a JSON string and sets the HTTP `Content-Type: application/json` header along with the appropriate status code.

Q38. What HTTP status code is returned when a user has no remaining rate limit?

HTTP 429: Too Many Requests.

Q39. What HTTP status code is returned when an unauthenticated user calls `upload-recording`?

HTTP 401: Unauthorized.

Q40. How is CORS handled in this project?

Because the Next.js frontend and API routes reside on the exact same domain origin, standard browser requests are same-origin, completely eliminating CORS configuration issues.

Q41. What does the `generate-followup-question` endpoint do?

It receives the current question and the candidate's spoken answer, calls the Google Gemini 1.5 Flash API, and returns an adaptive probing follow-up question.

Q42. What does the `delete-recording` endpoint do?

It verifies user ownership and permanently removes the video file from Supabase storage or server disk storage.

Category 5: Database & Convex BaaS

Q43. What is Convex?

Convex is a reactive serverless document database and backend-as-a-service. It synchronizes data in real time with the React frontend over persistent WebSockets.

Q44. What are the two main tables in your Convex schema?

`UserTable` (storing user profile name, email, avatar) and `InterviewSessionTable` (storing questions, answers, feedback, and video recording URLs).

Q45. How are the two tables linked?

Through a foreign key: `userId: v.id('UserTable')` in `InterviewSessionTable` points directly to the `_id` of the user in `UserTable`.

Q46. What is the difference between a Convex Query and Mutation?

A Query is a read-only, cached, reactive function that returns data (e.g., `GetInterviewQuestions`). A Mutation is a transactional function that writes or updates records in the database (e.g., `SaveInterviewQuestion`).

Q47. How does Convex achieve real-time synchronization?

Through WebSockets. When a client calls `useQuery`, Convex establishes a subscription. Whenever a mutation updates a record, Convex automatically pushes the new data to all active clients without HTTP polling.

Q48. Where are the Convex backend functions defined in the codebase?

Inside the `convex/` directory: `convex/schema.ts` (schema), `convex/Interview.ts` (interview logic), and `convex/users.ts` (user logic).

Q49. How do you prevent unauthorized users from viewing someone else's interview?

In `convex/Interview.ts`, `GetInterviewQuestions` verifies that `record.userId === args.uid`. If unauthorized, it logs a warning and returns `null`.

Q50. How is an interview session created in Convex?

By calling `await saveInterviewQuestion({ questions, resumeUrl, uid, jobTitle, ... })` which inserts a row into `InterviewSessionTable` with `status: 'draft'`.

Q51. How is interview feedback updated in the database?

Via the `UpdateFeedback` mutation in `convex/Interview.ts`, which patches the session record with the evaluation JSON and marks `status: 'completed'`.

Q52. Does Convex use SQL or NoSQL?

It is a document-based database (similar to NoSQL) but provides relational integrity through strongly typed foreign keys and strict TypeScript schema validators (`v.string()`, `v.id()`).

Q53. How do you generate Convex client types?

Running `npx convex dev` automatically generates client types into `convex/_generated/` whenever `schema.ts` is modified.

Q54. How do you inspect database records during development?

By running `npx convex dashboard`, which opens a visual database table browser in the web browser.

Category 6: Artificial Intelligence & Prompt Engineering

Q55. Which AI model is used in this project?

Google Gemini 1.5 Flash (`gemini-1.5-flash`).

Q56. Why use Gemini 1.5 Flash instead of Gemini 1.5 Pro or GPT-4?

Gemini 1.5 Flash is optimized for high-speed, low-latency conversational tasks. It responds in under 800ms, making it ideal for real-time interview pacing, while maintaining a generous free-tier API quota.

Q57. How is Gemini called in the code?

Via an Axios HTTP POST request in `lib/roleInterviewerEngine.ts` to `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${apiKey}`.

Q58. How do you enforce that Gemini returns valid JSON?

The system prompt explicitly commands: *"Respond with ONLY a raw JSON object (no markdown, no backticks)"*. The code also applies a regex cleanup (`replace(/```json/g, '').replace(/```/g, '')`) before calling `JSON.parse()`.

Q59. What happens if Gemini API call times out or fails?

The Axios call has a 7000ms timeout. If it errors, a `catch` block catches the exception and falls back to a deterministic, keyword-matching role intelligence engine. The interview never freezes.

Q60. How does the AI handle candidates who say "I don't know"?

The prompt specifically checks for struggle phrases like "I don't know". Gemini responds with a gentle pivot question: *"No worries! Let's shift our focus to another area..."* to keep candidate confidence high.

Q61. What are the 13 engineering domains recognized by the engine?

Java Developer, Python Developer, Frontend Developer, Backend Developer, Fullstack Developer, AI/ML Engineer, Data Scientist, Data Analyst, DevOps Engineer, Cloud Engineer, QA Engineer, Cybersecurity Engineer, and General Software Engineer.

Q62. How does the engine generate questions for unknown or custom roles?

`normalizeRole()` analyzes the combined role and tech stack string. If no specialized domain matches, it defaults to `software_engineer` and dynamically injects the candidate's exact tools into progressive scenario templates.

Q63. Does the post-interview scoring use Gemini?

No. Post-interview feedback is computed using a deterministic text-matching and regex algorithm in `app/api/interview-feedback/route.tsx`. This guarantees zero hallucinations in scoring.

Q64. How are verbal filler words counted?

Using regular expressions that match conversational fillers like `\b(um|uh|like|you know|basically|actually)\b` across the transcribed answer array.

Category 7: Audio, Video & Computer Vision

Q65. How does the application access the user's camera and microphone?

Using `navigator.mediaDevices.getUserMedia({ video: true, audio: true })`. The browser prompts the user for permission.

Q66. Which API records the interview video?

The browser `MediaRecorder` API. It records chunks of video in WebM format (`video/webm; codecs=vp8,opus`).

Q67. Why use IndexedDB for local video buffering?

Video files are large (20MB-100MB). LocalStorage is limited to 5MB and blocks the main thread. IndexedDB allows asynchronous, persistent storage of large binary Blobs, ensuring the video is safe even if the network drops.

Q68. How does the Canvas 2D posture tracking work?

Every 1000ms, a hidden 320×240 canvas draws the video frame, samples pixel RGBA values, detects skin-color centroids, and measures whether the candidate's face is centered or tilted.

Q69. How does the app check if lighting is adequate?

It calculates perceived luminance across sampled pixels: `0.299*R + 0.587*G + 0.114*B`. If average luminance is below 45, it flags "Too Dark"; if above 210, it flags "Overexposed".

Q70. How does synchronized question playback work?

When each question starts, the current video recording elapsed time (in seconds) is saved to `questionTimestamps`. On the playback screen, clicking a question sets `video.currentTime = timestamp`, instantly jumping to that answer.

Category 8: Authentication, Security & Deployment

Q71. How are private routes protected in Next.js?

Via `middleware.ts` using Clerk's `clerkMiddleware()`. It intercepts requests to `/dashboard` and `/interview` and forces an automatic redirect to `/sign-in` if no valid JWT session token exists.

Q72. What is Arcjet and how is it used?

Arcjet is a security SDK that provides bot detection and rate limiting. It uses a token-bucket algorithm to protect expensive API endpoints from abuse.

Q73. Why should API keys never be stored in GitHub?

Public repositories are scanned by bots within seconds. Leaked keys can result in stolen user data, massive cloud billing charges, or account suspension.

Q74. What is the purpose of `.env.example`?

It provides a safe, shareable template showing which environment variable names are required without revealing actual secret keys.

Q75. How do you deploy this application to production?

Push the code to GitHub, deploy the Convex backend using `npx convex deploy`, import the repo into Vercel, configure production environment variables, and update Clerk redirect URLs.

Q76. What is Continuous Deployment (CD)?

Whenever you run `git push origin main`, Vercel automatically detects the new commit, runs `npm run build`, and updates the live production site without manual server intervention.

Q77. What happens if an uploaded video exceeds Vercel's 4.5MB serverless payload limit?

The app supports direct cloud upload via signed URLs (Convex / Supabase) bypassing the Vercel serverless function payload limit entirely.

Q78. What is the difference between client-side validation and server-side validation?

Client-side validation (in React) provides instant user feedback (e.g., "Field cannot be empty"). Server-side validation (in API route) is essential for security because malicious users can bypass frontend forms by sending direct curl/HTTP requests.

Q79. How is candidate privacy protected regarding webcam recordings?

Recordings are stored only under the candidate's authenticated user ID. In `api/delete-recording`, candidates can permanently delete their cloud video files at any time.

Q80. What would you do if a database query is running slow?

Add database indexes on frequently queried fields (such as `userId` in `InterviewSessionTable`) to reduce table scan time from O(N) to O(log N).

CHAPTER 19: FINAL REVISION & QUICK REFERENCE

19.1 One-Page Executive Summary

AI Mock Interview 2.0 (MAPD) is an end-to-end full-stack web application developed with **Next.js 15.4 (App Router)**, **React 19**, **TypeScript**, and **Convex Reactive Document DB**.

The system addresses the high cost and passive nature of traditional placement preparation. A candidate enters their target role, tech stack, and optional resume. The platform normalizes the role into one of 13 engineering domains and generates an 8-phase structured technical interview. During the session, native browser **Web Speech API** handles voice dialogue, **HTML5 Canvas 2D** monitors posture and eye-contact stability, and **Google Gemini 1.5 Flash** provides real-time adaptive follow-ups. The full session is buffered in **IndexedDB**, uploaded to cloud storage, and evaluated deterministically against technical accuracy, verbal filler words, and the STAR framework.

19.2 The 2-Minute Viva Pitch Script

> 🎙️ **Read this aloud during your project presentation:**
>
> *"Good morning, examiners. Today I am presenting **MAPD AI Mock Interview 2.0**, a full-stack platform designed to prepare candidates for engineering placements.*
>
> *Most students practice interviews passively by memorizing textbook answers, which fails to prepare them for speaking under pressure. Our application simulates a real hiring interview.*
>
> *The candidate enters their target job title, tech stack, and uploads a resume. Our backend normalizes the role across 13 engineering domains and constructs an 8-phase structured interview — from fundamentals to architecture and behavioral scenarios.*
>
> *When the interview begins, the browser's native Web Speech API speaks the questions aloud and transcribes the candidate's voice. As they speak, an in-memory Canvas 2D engine samples video frames to measure posture and head centering. If the candidate mentions specific tools or struggles, our system calls Google Gemini 1.5 Flash to ask an adaptive follow-up question.*
>
> *Video recordings are preserved in browser IndexedDB for offline resilience and uploaded to cloud storage. Finally, our feedback engine evaluates the transcript, calculates verbal filler metrics, and provides model STAR answers.*
>
> *The entire project is built with Next.js 15, React 19, TypeScript, Clerk Authentication, and Convex Reactive Database. Thank you, and I am ready for your questions."*

19.3 Master Technology Cheat Sheet

Layer	Primary Tool	Key File	One-Sentence Explanation
Framework	Next.js 15.4	`app/layout.tsx`	Unifies React SSR frontend and serverless API endpoints in one codebase.
UI Library	React 19	`start/page.tsx`	Manages component rendering, state hooks, and virtual DOM.
Language	TypeScript 5	`tsconfig.json`	Enforces static type contracts and prevents runtime undefined crashes.
Database	Convex	`convex/schema.ts`	Serverless reactive document database streaming live updates via WebSockets.
Auth	Clerk	`middleware.ts`	Manages JWT cookies, OAuth sessions, and protects private route paths.
AI LLM	Gemini 1.5 Flash	`roleInterviewerEngine.ts`	Ultra-fast generative AI model generating real-time follow-up probes.
Storage	IndexedDB + Supabase	`recordingStorage.ts`	Zero-loss client video buffering with multi-tier cloud upload fallback.
Vision	Canvas 2D API	`facialAnalysis.ts`	Samples webcam pixels to monitor posture and eye-contact stability.

— END OF MASTER PROJECT DOCUMENTATION & VIVA GUIDE —